

Analyse approfondie du fichier algo.py : Le Cerveau du Système 🧠

Ce fichier contient la classe PortfolioBrain. À première vue, il s'agit d'une simple classe Python, mais elle incarne en réalité une couche d'intelligence système critique. Son but unique est de répondre à une question apparemment simple, mais qui tourmente les architectes réseaux depuis les années 80 : **"Combien de temps dois-je attendre une réponse avant de déclarer qu'il y a un problème ?"**

Dans les systèmes distribués, cette question est un piège mortel :

1. **Si on attend trop peu (Timeout agressif)** : On coupe des connexions valides qui étaient juste un peu lentes. Cela provoque des erreurs artificielles et incite le système à relancer la requête, aggravant la surcharge du réseau (c'est l'effet "boule de neige").
2. **Si on attend trop longtemps (Timeout laxiste)** : L'utilisateur s'énerve devant un écran figé. Pire, les threads du serveur restent bloqués en attente, consommant de la RAM inutilement, ce qui peut mener à un crash complet du serveur sous forte charge.

Cet algorithme trouve l'équilibre parfait en s'adaptant en temps réel, transformant un paramètre statique (le timeout) en une variable vivante.

1. Contexte et Configuration (Constantes)

Le début de la classe ne se contente pas de définir des variables ; il définit les limites physiques et logiques de votre environnement d'exécution. Chaque constante est une réponse à une contrainte du monde réel.

```
class PortfolioBrain:  
    # ...  
    TIMEOUT_MIN = 1.0 # Plancher de sécurité  
    TIMEOUT_MAX = 10.0 # Plafond UX  
    MEMORY_TTL = 600 # 10 minutes avant "l'oubli"
```

Analyse "Senior" et Justifications :

- **TIMEOUT_MIN = 1.0 (Le Bouclier Anti-Bruit) :**
 - *Le Problème Technique* : Vous hébergez votre site sur le **Free Tier de Render**. C'est un environnement "multi-tenant" (colocation), où votre code partage le même processeur (CPU) que des centaines d'autres applications.
 - *Le Phénomène de "Throttling"* : Si un voisin devient bruyant, l'hyperviseur peut

geler votre application pendant 200ms ou 500ms pour redonner la main aux autres.

- *La Conséquence* : Si votre timeout était réglé techniquement à 0.3s (suffisant pour une requête HTTP normale), ce gel CPU causerait un timeout. Vous penseriez à une panne réseau, alors que c'est juste le CPU qui a "hiccupé".
- *La Solution* : Ce plancher de 1.0s agit comme un filtre passe-bas. Il ignore ces micro-gels CPU pour éviter les **Faux Positifs**.
- **TIMEOUT_MAX = 10.0 (La Limite Physiologique)** :
 - *La Règle de Doherty (UX)* : Au-delà de quelques secondes, l'attention de l'utilisateur décroche. Au-delà de 10 secondes, l'utilisateur considère que le système est cassé et recharge la page ou s'en va.
 - *Gestion des Ressources Serveur* : Un thread Unicorn (le worker web) est une ressource précieuse. Le bloquer pendant 30 ou 60 secondes sur une requête morte est du gaspillage. Mieux vaut "échouer vite" (Fail Fast) à 10s pour libérer le worker pour quelqu'un d'autre.
- **MEMORY_TTL = 600 (La Gestion du "Cold Start")** :
 - *Le Contexte FaaS/PaaS* : Les serveurs modernes ne tournent pas en continu à plein régime. Ils s'endorment ou changent de contexte.
 - *L'Obsolescence de la Donnée* : Une moyenne de latence calculée à 08h00 (heure creuse, réseau rapide) est-elle valable à 18h00 (heure de pointe Netflix, réseau saturé) ? Non.
 - *La Stratégie* : Après 600 secondes (10 minutes) sans activité, on considère que notre modèle statistique est périmé. Il faut le "dégrader" (voir section *Soft Decay*) pour ne pas prendre de décision basée sur des fantômes du passé.

2. L'Initialisation et la Sécurité (__init__)

```
def __init__(self):
    self._lock = threading.Lock()

    self.srtt = self.DEFAULT_TIMEOUT # Moyenne lissée (3.0s au début)
    self.rttvar = 0.5                # Variation/Incertitude
```

Le threading.Lock() : Garantie d'Intégrité (Thread Safety)

- **Le Scénario de la "Race Condition"** :
 - Flask (avec Unicorn en mode gthread) traite plusieurs requêtes en parallèle dans le même processus mémoire.
 - Imaginez deux utilisateurs, Alice et Bob, qui visitent le site exactement au même moment.
 - Le thread d'Alice lit self.srtt, commence un calcul... mais avant qu'elle n'écrive le

résultat, le thread de Bob lit l'ancienne valeur, fait son calcul, et écrase le travail d'Alice.

- *Résultat* : Votre modèle mathématique devient corrompu et incohérent.

- **La Solution "Section Critique" :**

- Le Lock implémente une **exclusion mutuelle**. C'est comme la clé unique des toilettes. Si Alice entre (acquiert le lock) pour mettre à jour les stats, Bob doit attendre à la porte.
- Cela garantit l'**atomicité** des opérations de mise à jour. C'est indispensable quand on manipule des statistiques partagées en mémoire.

Les Variables d'État (L'Héritage TCP)

- **srtt (Smoothed Round Trip Time) :**

- Ce n'est pas une simple moyenne arithmétique. C'est une moyenne pondérée exponentielle. Elle représente "l'attente normale" actuelle.

- **rttvar (Round Trip Time Variation) :**

- C'est la mesure de la **gigue** (Jitter).
- *Réseau Stable* : Latences de 100ms, 101ms, 99ms. rttvar est proche de 0.
- *Réseau Chaotique (Wifi public, 4G instable)* : Latences de 50ms, puis 800ms, puis 200ms. rttvar explose.
- Plus rttvar est haut, plus l'algorithme devient prudent et augmente le timeout.

3. Le Calcul Mathématique (_calc_timeout_unsafe)

C'est ici que réside la formule magique de Van Jacobson (1988), qui a sauvé Internet de l'effondrement dû à la congestion.

```
def _calc_timeout_unsafe(self):  
    # Formule : Moyenne + 4 * Variation  
    rto = self.srtt + (4 * self.rttvar)  
    return max(self.TIMEOUT_MIN, min(self.TIMEOUT_MAX, rto))
```

Explication Approfondie de la Formule :

- **Pourquoi Moyenne + Variation ?**

- Si on réglait le timeout exactement sur la moyenne, 50% des requêtes échoueraient (toutes celles qui sont légèrement au-dessus de la moyenne). Il faut une marge de sécurité.

- **Pourquoi le multiplicateur 4 (Règle des 4 Sigmas) ?**

- C'est une application pratique de l'inégalité de Chebyshev (ou Tchebychev).

- En statistique, si la distribution des latences suivait une courbe en cloche parfaite (Gaussienne), 3 écarts-types suffiraient pour couvrir 99.7% des cas.
- Mais les latences réseaux ne sont **pas** Gaussiennes. Elles ont une "longue traîne" (il y a souvent des paquets très très lents).
- Le coefficient **4** est une heuristique robuste choisie par les ingénieurs TCP pour couvrir 99.9% des scénarios, même les plus chaotiques, sans être trop pessimiste.
- **Le Clamp (max/min) :**
 - C'est un garde-fou ("Sanity Check"). Même si les maths disent "attends 0.001s" (impossible physiquement) ou "attends 2 heures" (inutile), on force le résultat à rester dans la fenêtre logique [1s, 10s] définie par nos constantes métier.

4. La Prédiction (get_timeout)

Cette fonction est appelée par app.py **AVANT** d'envoyer la requête. C'est une étape de prédiction basée sur l'historique.

```
def get_timeout(self):
    with self._lock: # On verrouille !
        time_since_last = time.time() - self.last_request_time

    if time_since_last > self.MEMORY_TTL:
        # STRATÉGIE "SOFT DECAY"
        self.rttvar = max(self.rttvar * 2, 1.0)
        # ... Recalcul ...
```

La Stratégie du "Soft Decay" (Dégradation Douce) :

- **Le Problème de l'Amnésie :** Si le serveur s'est endormi pendant 2 heures, les données de latence qu'il a en mémoire sont vieilles.
 - *Option A (Reset Total) :* On remet tout à zéro (3s par défaut). Risqué, car on perd la connaissance que ce service météo est peut-être structurellement lent.
 - *Option B (Rien faire) :* On utilise la vieille valeur. Risqué, car le réseau a pu changer.
- **La Solution Hybride (Soft Decay) :**
 - On conserve la moyenne (srtt) telle quelle (c'est notre meilleure estimation de base).
 - Mais on **double l'incertitude** (rttvar * 2).
 - *Analogie :* "Je me souviens que le trajet prenait 20 minutes (Moyenne conservée), mais comme je n'y suis pas allé depuis un an, je vais prévoir une marge de sécurité énorme (Variation doublée) au cas où il y ait des travaux."
 - Cela permet une reprise en douceur, très prudente, avant de raffiner à nouveau le

modèle.

5. L'Apprentissage (update)

Cette fonction est appelée **APRÈS** la requête. C'est le cœur de l'apprentissage machine (au sens algorithmique strict).

```
def update(self, observed_latency, success: bool):
    with self._lock:
        if not success:
            # --- ALGORITHME DE KARN (PUNITION) ---
            self.current_timeout = min(self.TIMEOUT_MAX, self.current_timeout * 2)
            self.rttvar += 0.5
        else:
            # --- ALGORITHME DE JACOBSON (LISSAGE) ---
            diff = observed_latency - self.srtt
            self.srtt = self.srtt + (0.125 * diff)
            self.rttvar = self.rttvar + (0.25 * (abs(diff) - self.rttvar))
```

Cas 1 : Échec / Timeout (Algorithme de Karn)

- **L'Ambiguïté de la Retransmission** : Quand un timeout survient, on ne sait pas si le paquet a été perdu ou s'il était juste très lent.
- **La Règle de Karn** : "N'apprends jamais d'un échec." Si on intégrait le temps du timeout dans la moyenne, on fausserait les stats.
- **Le Backoff Exponentiel** :
 - Action : $\text{current_timeout} * 2$.
 - On passe de 1s -> 2s -> 4s -> 8s -> 10s (max).
 - C'est vital pour éviter la **Congestion Collapse**. Si le réseau est saturé, la pire chose à faire est de réessayer vite et souvent. Il faut s'écarter et laisser le réseau "respirer".

Cas 2 : Succès (Algorithme de Jacobson)

- On a réussi en `observed_latency` (ex: 0.2s).
- **Lissage Alpha (0.125) ou Moyenne Mobile Exponentielle (EWMA)** :
 - $\text{srtt} = \text{srtt} + 0.125 * (\text{nouveau} - \text{ancien})$
 - Cela signifie que la nouvelle moyenne est composée de **87.5% de l'histoire passée** et **12.5% de la mesure actuelle**.
 - *Pourquoi ?* Cela agit comme un amortisseur. Si une seule requête est bizarrement lente (un "spike"), la moyenne globale ne va pas exploser. Elle va monter doucement. Cela rend le système **stable**.

- **Lissage Beta (0.25) :**
 - On met à jour l'incertitude (rttvar) avec un poids de 25%. On apprend plus vite sur la variation que sur la moyenne, ce qui permet de réagir rapidement si le réseau devient instable.

6. L'Observabilité (get_stats)

```
def get_stats(self):
    # Retourne srtt, rttvar, timeout...
```

- **Le "Black Box" Problem :** Sans cette méthode, votre algorithme est une boîte noire. Vous ne savez pas s'il marche.
- **Debugging en Production :** En affichant ces stats dans les logs (via app.py), vous pouvez corrélérer des incidents.
 - *Exemple :* "Tiens, à 14h00, le rttvar a triplé et le timeout est monté à 8s. C'est corrélé avec la panne d'Amazon AWS dans la région us-east-1."
 - C'est la différence entre dire "ça a planté" et "la latence réseau a dégradé le service".

Résumé Architectural

Le fichier algo.py est une démonstration de force technique. Il effectue une transposition pure des mécanismes de la couche **Transport (TCP)** — normalement gérés par le noyau du système d'exploitation — vers la couche **Application (HTTP)**.

1. **Il rend l'application organique :** Elle "ressent" l'état du réseau et s'adapte, au lieu d'appliquer une règle bête et méchante.
2. **Il optimise les ressources économiques et techniques :** Il coupe vite les connexions mortes (CPU saving) mais sait être patient quand le réseau est juste un peu lent (User Experience saving).
3. **Il démontre une culture "Computer Science" :** L'utilisation explicite de verrous (Lock) pour la sûreté des threads et l'implémentation d'algorithmes historiques (Jacobson/Karn 1988) prouve que vous maîtrisez les fondements théoriques de l'informatique, bien au-delà de la simple utilisation de frameworks Web modernes.